

RouteWise: Latency-Cost Optimization for Multi-Provider LLM Routing

Harvard MadSys Lab

April 3, 2026

Outline

- 1 Motivation & Problem
- 2 System Design: Cost Optimization
- 3 System Design: Latency Optimization
- 4 Implementation
- 5 Evaluation
- 6 Discussion & Conclusion

Cost Challenge: Hybrid Pricing & Opportunity Cost

The Problem: LLM inference cost is a critical barrier—API bills rival engineering salaries for startups.

The Opportunity: Commoditization of open-weight models means the *same model* is served under radically different pricing:

Type	Symbol	Constraint	API	Local GPU
Pay-per-token	S_A	Unlimited; \$/tok	Together	N/A
Quota sub.	S_Q	$\leq Q/\text{window}$	Chutes.ai	Gutter pool
Concurrency	S_C	$\leq C$ parallel	Featherless	GPU cluster

S_Q, S_C : **zero marginal cost** up to capacity.

Unified abstraction—grounded in **physical resource constraints**, not just API pricing.

Opportunity Cost Challenge

Greedy/FCFS wastes scarce subscription slots:

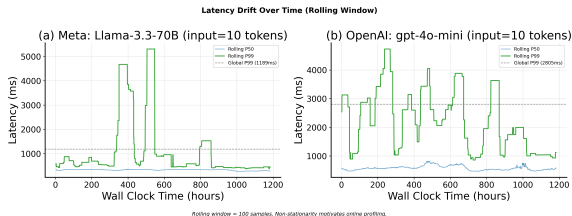
- “Hello” → \$0.001 via API
Wastes a quota slot
- Coding task → \$0.500 via API
Ideal for subscription

On real workloads, Greedy captures only **49–83%** of optimal savings.

Goal: Route high-value requests to subscriptions under uncertainty.

Latency Challenge: Provider Heterogeneity & Drift

The Problem: Interactive apps need strict latency SLOs, but provider latencies are **non-stationary**.



P99 latency over 1,200 hours: Meta Llama spikes 1.2s→5.4s,
OpenAI GPT-4o-mini varies 1–4.8s

Provider	P50	P99	Cost/1K
Groq	382ms	796ms	\$0.69
SambaNova	378ms	1.4s	\$0.68
Parasail	455ms	4.2s	\$0.21
Together	634ms	15.7s	\$0.88

- P99 varies **20x** across providers
- P99 fluctuates **5x** over time
- Fast providers cost more; cheap ones have high tails

Challenge

Minimize cost while guaranteeing latency SLO—static selection is inadequate.

Core Question:

Given a stream of LLM requests with heterogeneous costs and latency requirements,
how should we route each request across hybrid providers
to **minimize cost** while **satisfying latency SLOs**?

Cost Routing

- **Problem:** Allocate requests across S_A , S_Q , S_C
- **Challenge:** **Opportunity cost** of scarce subscriptions
- **Solution:** Primal-dual threshold strategy

Latency Routing

- **Problem:** Select among 12+ providers per request
- **Challenge:** **Non-stationary** latency profiles
- **Solution:** LP-based mixing + smart hedging

Online Setting

- **Problem:** **Irrevocable** sequential decisions
- **Challenge:** Unknown output tokens + drift
- **Solution:** Learning-augmented online algorithms

Quota-Constrained Routing ($S_Q + S_A$)

Problem Setup:

- Providers: S_Q (quota) + S_A (API)
- Q requests/day on S_Q (zero cost)
- API cost: $c_i = \frac{n_i^{\text{in}}}{1000} p^{\text{in}} + \frac{n_i^{\text{out}}}{1000} p^{\text{out}}$

Key Assumption: Fixed vs. Rolling

- Real subscriptions often use **rolling windows**.
- We approximate this via **disjoint fixed intervals** (e.g., calendar day) where quota resets entirely.
- Yields an **optimistic lower bound** on API cost.

Optimization:

$$\begin{aligned} \min \quad & \sum_{i \in \mathcal{R}} x_i^A \cdot c_i^{\text{API}} \\ \text{s.t.} \quad & \sum_{i: \text{day}(a_i)=d} x_i^Q \leq Q, \quad \forall d \end{aligned}$$

Optimal Algorithm

Closed-form solution:

- 1 For each day d , collect $\mathcal{R}_d$
- 2 Sort by c_i^{API} descending
- 3 Top- $Q \rightarrow S_Q$
- 4 Remainder $\rightarrow S_A$

Complexity: $O(n \log n)$

Intuition: Requests effectively “bid” for scarce quota using their avoided API cost.

Joint Quota-Concurrency Routing ($S_Q + S_C + S_A$)

Additional Provider:

- S_C : Concurrency-limited subscription
- At most C requests executing **simultaneously**
- Each request i occupies a slot for p_i seconds

New Constraints:

$$\text{(Concurrency)} \quad \sum_i \mathbf{1}[s_i \leq t < s_i + p_i] \cdot x_i^C \leq C$$

$$\text{(Arrival)} \quad s_i \geq a_i$$

$$\text{(Deadline)} \quad s_i + p_i \leq d_i$$

Complexity Result

Joint routing is **NP-hard**

Couples:

- Knapsack (daily quota)
- Parallel machine scheduling
- Release times + deadlines

Our Solution

Time-Indexed ILP

- Discretize into slots
- Use MIP solver (Gurobi)
- Exact optimal solution

Joint Routing: ILP Formulation

Time Discretization:

- Horizon: $t \in \{0, 1, \dots, T\}$ with slot width Δ
- Processing time: $\hat{p}_i = \lceil p_i / \Delta \rceil$ slots
- Valid window: $[\alpha_i, \beta_i]$ based on arrival and deadline

Variables:

- $x_i^Q, x_i^C, x_i^A \in \{0, 1\}$: routing
- $z_{i,t} \in \{0, 1\}$: start at slot t

Objective:

$$\min \sum_{i \in \mathcal{R}} x_i^A \cdot c_i^{\text{API}}$$

Key Constraints:

$$x_i^Q + x_i^C + x_i^A = 1$$

$$\sum_{t=\alpha_i}^{\beta_i} z_{i,t} = x_i^C$$

$$\sum_i \sum_{\tau} z_{i,\tau} \leq C \quad \forall t$$

Online Routing: Challenges & Greedy Baseline

Online Setting: Requests arrive sequentially; routing decisions are **irrevocable**.

Known:

- Model type m_t
- Input tokens n_t^{in}
- Remaining quota q_t

Unknown:

- **Output tokens** n_t^{out}
- **Future requests**

Algorithm 1 Greedy Strategy

```
1: if  $q_t > 0$  then
2:   return SUBSCRIPTION
3: else
4:   return CHEAPEST_API( $r_t$ )
5: end if
```

Cost-Aware Greedy Variant

```
1:  $\hat{c}_t \leftarrow \text{EstimateCost}(r_t)$ 
2:  $\theta \leftarrow \frac{\text{Monthly Fee}}{30 \times Q}$ 
3: if  $\hat{c}_t > \theta$  and  $q_t > 0$  then
4:   return SUBSCRIPTION
5: else
6:   return API
7: end if
```

Greedy Limitations

- Always uses quota first (FCFS)
- **Wastes quota on cheap requests**
- Captures only **49–83%** of optimal savings

Primal-Dual Threshold Strategy

Core Insight: Quota is a **scarce resource**—assign it an **Opportunity Cost**!

Shadow Price = Opportunity Cost

The threshold $\psi(z)$ represents the **marginal value** of the next quota slot:

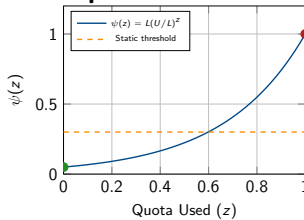
$$\psi(z) = L \cdot \left(\frac{U}{L}\right)^z$$

- $z = k/Q$: fraction of quota consumed
- L, U : min/max request value bounds
- $\psi(z) \uparrow$ as quota becomes scarcer

Decision Rule:

$$\text{Route to } S_Q \iff \underbrace{\hat{v}_t}_{\text{value}} \geq \underbrace{\psi(z)}_{\text{opp. cost}} \text{ and } k < Q$$

Adaptive Threshold



Theoretical Guarantee

Competitive ratio: $O(\ln(U/L) + 1)$
(Optimal for online knapsack with prices)

Primal-Dual: Handling Unknown Output Length

The Reality: Output length is unknown at routing time!

- We cannot compute true value v_t
- We must predict $\hat{v}_t$ using historical distributions

Output Length Predictors

- **Histogram:** Maintains full empirical CDF.
Pros: Well-calibrated (accurate P50/P10).
Cons: Slow to adapt to workload shifts.
- **EMA (Exponential Moving Average):**
$$\hat{n}_{\text{new}} = \alpha \cdot n_t + (1 - \alpha) \cdot \hat{n}_{\text{old}}$$

Pros: Adapts in ~ 10 samples.
Cons: Point estimate, statistical bias.

Routing with Predictions

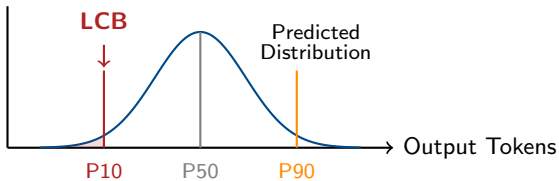
```
1:  $\hat{n}_t^{\text{out}} \leftarrow \text{Predictor.predict}()$ 
2:  $\hat{v}_t \leftarrow \text{Price}_{\text{in}} \cdot n_t^{\text{in}} + \text{Price}_{\text{out}} \cdot \hat{n}_t^{\text{out}}$ 
3:  $\theta \leftarrow \psi(k/Q)$  {Shadow Price}
4: if  $\hat{v}_t \geq \theta$  and  $k < Q$  then
5:   return SUBSCRIPTION
6: else
7:   return API
8: end if
```

Risk: What if EMA over-predicts length?
We waste scarce quota on a cheap request!

Learning-Augmented Variant (LA)

Recall: PD thresholds on median estimate $\hat{v}$ — may over-accept when predictor is noisy.

LA variant: Route based on **Conservative Value Estimates**.



LCB (Lower Confidence Bound): Use P10 to avoid over-estimating value $\rightarrow$ prevents quota waste.

Risk-Aware Decision Rule

- 1: $\hat{v}_{\text{LCB}} \leftarrow \text{Predictor.P10}(r_t)$
- 2: $\theta \leftarrow \psi(k/Q)$ {Shadow Price}
- 3: **if** $\hat{v}_{\text{LCB}} \geq \theta$ **and** $k < Q$ **then**
- 4: **return** SUBSCRIPTION
- 5: **else**
- 6: **return** API
- 7: **end if**

Why It Works

- **High Variance?** $\text{LCB} \ll \text{Median}$
 $\rightarrow$ Uncertain value $\rightarrow$ Use API (safe)
- **Low Variance?** $\text{LCB} \approx \text{Median}$
 $\rightarrow$ Confident value $\rightarrow$ Use S_Q

Worked Example: Primal-Dual vs Greedy

Setup: Quota $Q = 3$, value bounds $[L, U] = [\$0.10, \$1.00]$. Five requests arrive:

Req	Out Tok	Cost
r_1	200	\$0.15
r_2	800	\$0.45
r_3	100	\$0.08
r_4	1500	\$0.85
r_5	600	\$0.38

Greedy: Accept in order $\rightarrow r_1, r_2, r_3$ to S_Q

API cost: $\$0.85 + \$0.38 = \$1.23$. Savings: $\$0.68$

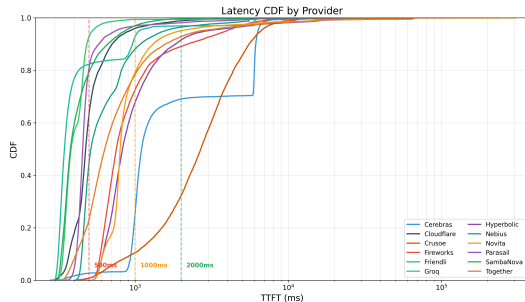
Primal-Dual: $\psi(z) = 0.10 \cdot 10^z$

- r_1 : $z=0$, $\psi=\$0.10$. $\$0.15 > \0.10 **Accept**
- r_2 : $z=0.33$, $\psi=\$0.22$. $\$0.45 > \0.22 **Accept**
- r_3 : $z=0.67$, $\psi=\$0.46$. $\$0.08 < \0.46 **Reject**
- r_4 : $z=0.67$, $\psi=\$0.46$. $\$0.85 > \0.46 **Accept**
- r_5 : Quota full $\rightarrow$ API

API cost: $\$0.08 + \$0.38 = \$0.46$. Savings: $\$1.45$

Primal-Dual saves 2.1x more by rejecting cheap r_3 (\$0.08) and reserving quota for expensive r_4 (\$0.85).

Provider Profiling & CDF Estimation



ECDF of TTFT across 12 providers

Data: llama-3.3-70b, 24h, **132,900** requests, 12 providers

Provider	P50	P99	Cost/1K
Groq	382ms	796ms	\$0.69
SambaNova	378ms	1.4s	\$0.68
Parasail	455ms	4.2s	\$0.21
Together	634ms	15.7s	\$0.88

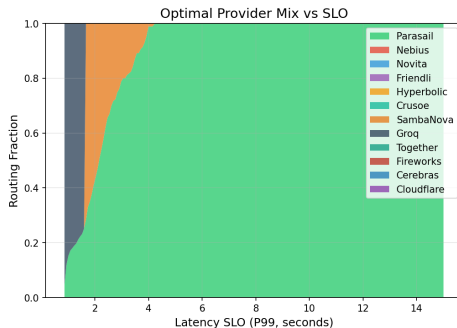
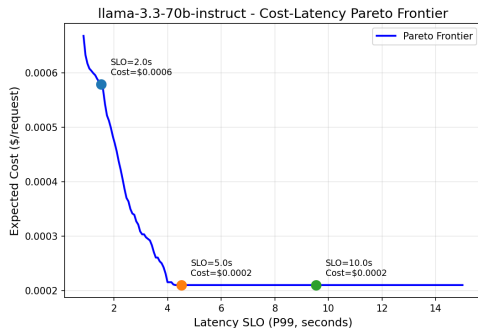
Key Insight: P99 latency varies **20x** across providers!

Implication:

- Fast providers (Groq) cost more
- Cheap providers (Parasail) have high tail latency
- **Opportunity for cost-latency tradeoff**

LP-Based Optimal Provider Mixing

Formulation: $\min_{\pi} \sum_j \pi_j c_j \quad \text{s.t.} \quad \sum_j \pi_j F_j(L) \geq 0.99, \quad \sum_j \pi_j = 1, \pi_j \geq 0$



Key Insight:

- $F_j(L)$ = empirical CDF at SLO L
- At most 2 providers at optimal (LP vertex)

Left: Cost-Latency Pareto Frontier

Right: Optimal provider mix vs SLO

Online Adaptive Routing

Challenge: Latency distributions are **non-stationary**

- Provider load varies by time-of-day
- Need **adaptive** profiling

Moving Window Profiler:

- Short (15 min) + Long (3h) windows
- Adaptive: $\beta = \frac{N_{\text{short}}}{N_{\text{short}} + \lambda}$

$$\hat{F}(L) = \beta \cdot F_{\text{short}} + (1 - \beta) \cdot F_{\text{long}}$$

Online Algorithm:

- 1 Update profiles with recent samples
- 2 Solve LP: $\pi^* = \arg \min_{\pi} \sum_j \pi_j c_j$
- 3 Sample provider via $\text{SWRR}(\pi^*)$

SWRR = Smooth Weighted Round-Robin

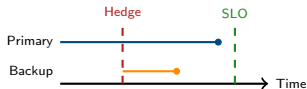
- Deterministic sampling from π^*
- Exact traffic proportions

Smart Hedging: Cost-Benefit Rule

Problem: Tail latency causes SLO violations even with optimal routing.

Cost-benefit rule: Hedge when expected benefit > backup cost:

$$\underbrace{\frac{S_{\text{primary}}(L)}{S_{\text{primary}}(t)}}_{\text{Primary Risk}} \times \underbrace{F_{\text{backup}}(\text{Rem})}_{\text{Backup Viability}} > \underbrace{\frac{C_b}{V}}_{\text{Cost Ratio}}$$



Key Properties

- **Mathematically Rigorous:** Evaluates joint probability that *both* models violate SLO.
- **Economically Interpretable:** Hedge only if expected benefit > backup cost.
- **Natural Adaptivity:** Automatically hedges more aggressively if backup is cheap ($C_b \downarrow$), and more conservatively if backup is expensive ($C_b \uparrow$).

Strategy	Viol.	Hedge	C_b/V
No hedging	1.4%	0%	—
Fixed 0.5	0%	5.2%	—
Survival	0%	15.2%	—
Economic	0%	8.6%	0.05

Implementation & Scalability

ILP Solver Performance (Gurobi):

Requests	Variables	Constraints	Time
100	10K	5K	0.5s
500	250K	25K	8s
1,000	1M	50K	45s
5,000	25M	250K	12min

Note: ILP is offline oracle only.
Production uses online algorithm.

Online Routing Latency:

Operation	Latency
EMA prediction	$< 1\mu s$
Histogram lookup	$\sim 10\mu s$
Threshold computation	$< 1\mu s$
Total routing decision	$< 50\mu s$

Negligible vs LLM inference latency (seconds).

Key: Online algorithm adds
 $< 50\mu s$ overhead per request

Experimental Setup

Datasets:

Dataset	Requests	Days	Models
ShareGPT	201K	7	1
FreeInference	371K	90	12
Enterprise	55K	84	8
BurstGPT	1.4M	61	1

Provider Configuration:

- S_Q : Daily quota, \$20/month
- S_A : Pay-per-token (multi-model pricing)

Strategies Compared:

- 1 **All-API**: Upper bound (no subscription)
- 2 **Greedy**: Use quota FCFS
- 3 **Optimal**: Offline oracle (sort by cost)

Online: 2×2 Design

	PD	LA
EMA	PD-EMA	LA-EMA
Histogram	PD-Hist	LA-Hist

Rows = output length predictor:

EMA: exponential moving average (Sec. 2)

Histogram: streaming quantile estimator (Sec. 2)

Cols = decision rule:

PD: threshold on $\hat{v}$ (median, Sec. 2)

LA: threshold on v^{LCB} (P10, Sec. 2)

Production Evaluation: 24h OpenRouter Experiment

Setup: 24h live evaluation against **OpenRouter API**.

Model: 11ama-3.3-70b, SLO=2s, 7,030 reqs/policy, all policies run simultaneously.

Policy	P50	P99	Viol.	Avg Cost
OpenRouter Auto	754	6,316	11.5%	\$0.009
Nebius (same cost)	446	5,858	11.2%	\$0.009
Cheapest (Parasail)	412	3,732	3.1%	\$0.008
Fastest (Groq)	269	1,048	5.3%	\$0.029
LP-Mix (Ours)	413	3,159	3.2%	\$0.009
Smart Hedge	381	1,293	0.6%	\$0.014

Cost per request (in \$/1K tokens). Nebius has similar cost to LP-Mix but 20x worse violation rate.

Smart Hedge vs OpenRouter Auto:

- **20x** fewer SLO violations (0.6% vs 11.5%)
- **4.9x** lower P99 latency (1.3s vs 6.3s)
- LP router **adapts** over 24h: Novita → Parasail (drift)

vs Nebius (same cost): Same cost, **0.6% vs 11.2%** violations. Mixing + hedging, not just picking the right provider.

vs Fastest (Groq): **2x** lower cost, **9x** fewer violations. Groq has 4.9% error rate.

Smart Hedge: best of both worlds—
lowest violations AND adaptive to provider drift

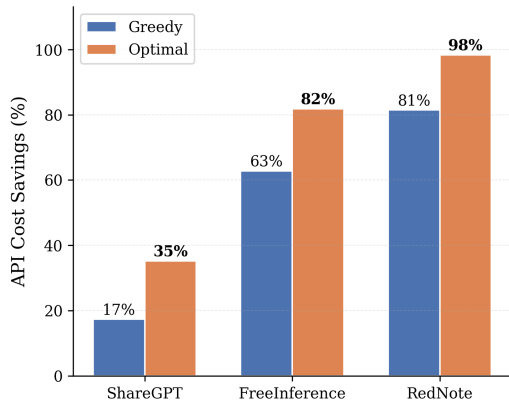
Offline Results: Quota-Constrained Routing

Dataset	Type	Greedy	Optimal
ShareGPT	single	17%	35%
FreeInference	multi	63%	82%
Enterprise	multi	81%	98%

Table: API cost savings vs All-API (DeepSeek-R1 pricing)

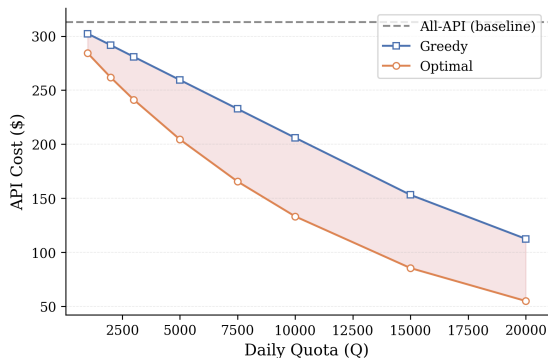
Key Observations:

- Optimal saves **35–98%** API costs
- Greedy leaves **18–20%** on the table
- Enterprise: High variance $\Rightarrow$ large gap

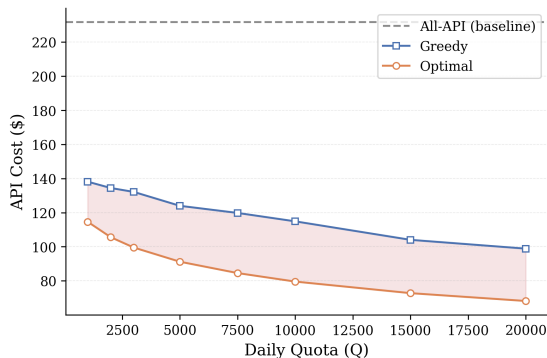


Takeaway: Optimal routing doubles the savings on some workloads!

Quota-Constrained: Sensitivity Analysis



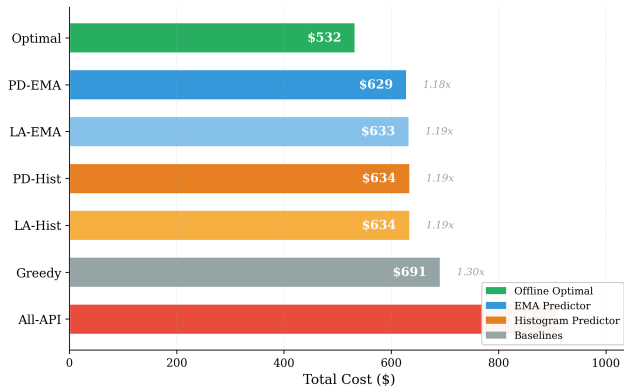
ShareGPT (single-model)



FreeInference (multi-model)

Key Insight: The gap between Greedy and Optimal **widens** as quota increases.
Shaded area = potential savings from optimal routing

Online Results: Cost Comparison



BurstGPT Dataset:

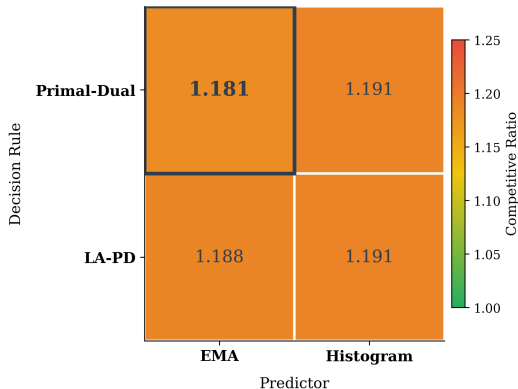
- 1.4M requests, 61 days
- DeepSeek-R1 pricing
- Daily quota $Q = 5000$

Strategy	Cost	CR
All-API	\$907	1.70x
Greedy	\$691	1.30x
PD-EMA	\$629	1.18x
LA-EMA	\$633	1.19x
Optimal	\$532	1.00x

30% savings vs All-API
9% better than Greedy

Ablation Study: Predictor vs Decision Rule

2x2 Ablation: Predictor vs Decision Rule



2x2 Factorial Design:
Predictor $\times$ Decision Rule

Key System Insights:

1. Predictor Dominance

Predictor quality > rule complexity.

- Switching Hist $\rightarrow$ EMA improves CR more than switching PD $\rightarrow$ LA.

2. The Power of Simplicity

EMA outperforms Histogram. Why?

- Faster adaptation to non-stationary arrivals
- Better data efficiency (fewer samples to warm up)

Lesson: In online systems,
lightweight adaptivity > complex modeling

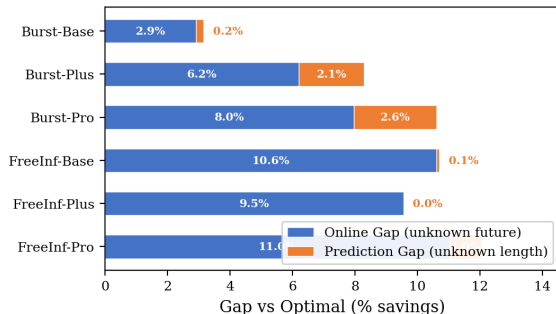
Do We Need Better Output Length Prediction?

Question: Literature proposes complex output length predictors (early-layer embeddings, fine-tuned models). **How much does misprediction actually hurt routing cost?**

Method: Same PD router with **Oracle** (true n_t^{out}) vs. **EMA** (online estimate).

Gap Decomposition:

$$\underbrace{S_{\text{Opt}} - S_{\text{EMA}}}_{\text{Total Gap}} = \underbrace{(S_{\text{Opt}} - S_{\text{Oracle}})}_{\text{Online Gap (unknown future)}} + \underbrace{(S_{\text{Oracle}} - S_{\text{EMA}})}_{\text{Prediction Gap (unknown length)}}$$



Dataset	Plan	EMA	Oracle (savings %)	Opt.	Pred. Gap
Burst.	Base	2.9	3.2	6.1	0.2
Burst.	Plus	15.9	18.0	24.2	2.1
Burst.	Pro	30.7	33.3	41.3	2.7
FreeInf.	Base	33.3	33.4	44.0	0.1
FreeInf.	Plus	45.9	45.6	55.2	-0.3
FreeInf.	Pro	45.0	46.1	57.1	1.0

Prediction Gap $\leq 2.7\%$

Knowing exact output length improves savings by **at most 2.7%**. The dominant gap is not knowing *future requests*.

Conclusion: Simple EMA is sufficient.
No need for embedding-based or fine-tuned predictors.

Online Joint Optimization (Quota+Concurrency)

Setup: FreeInference trace, 371K requests, 62 days
Single-model (deepseek-r1), $C = 8$, multiplier = 2
 S_Q : \$20/mo (5K/day) S_C : \$75/mo

Strategy	Total	API	CR
All-API	\$368	\$368	1.251×
S_Q -Only	\$172	\$112	0.586×
S_C -Only	\$461	\$236	1.570×
Greedy	\$345	\$60	1.174×
PrimalDual	\$332	\$47	1.128×
LA-PD	\$320	\$35	1.087×
ILP-Optimal	\$294	\$9	1.000×

Subscription cost: \$285 (S_Q \$60 + S_C \$225, prorated 62 days)

Key Observations:

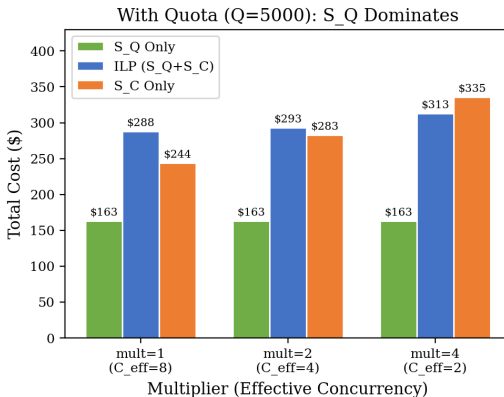
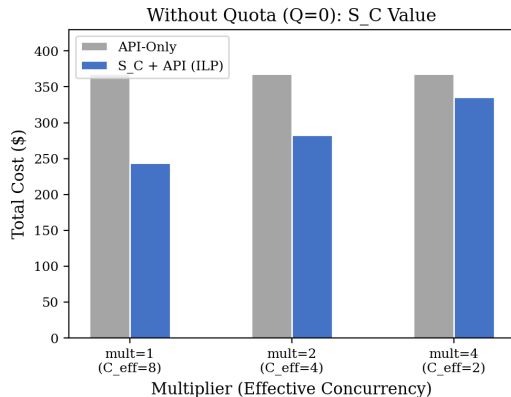
- Joint routing (ILP) saves **20%** vs. All-API
- S_Q -Only already saves 53%, but adding S_C pushes API cost from \$112 $\rightarrow$ \$9
- S_C -Only loses money (subscription > savings)

Online Algorithm Quality

Strategy	CR	% of Optimal Savings
Greedy	1.174×	31%
PrimalDual	1.128×	49%
LA-PD	1.087×	65%

Learning-augmented (LA-PD) captures **2×** more savings than Greedy.

Joint Optimization: Single-Model Comparison



Setup: FreeInference, deepseek-r1, $C=8$

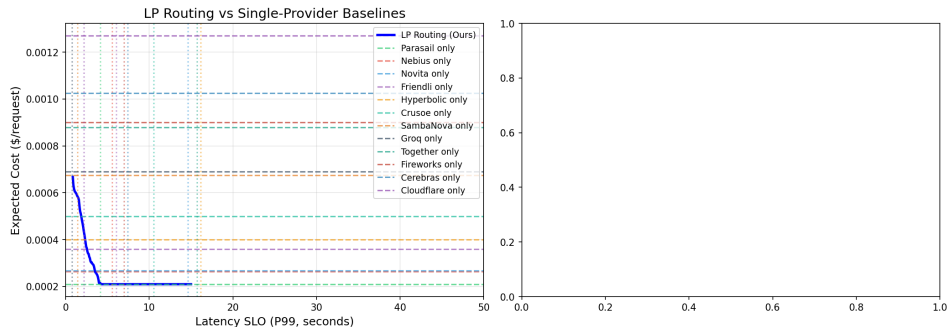
Multiplier controls effective concurrency

mult=1: $C_{eff}=8$ mult=2: $C_{eff}=4$ mult=4: $C_{eff}=2$

Takeaway:

- Larger models $\rightarrow$ lower S_C value
- S_Q alone covers 83% of requests
- S_C adds value only when S_Q unavailable

Phase 2 Results: Optimal Routing Decisions



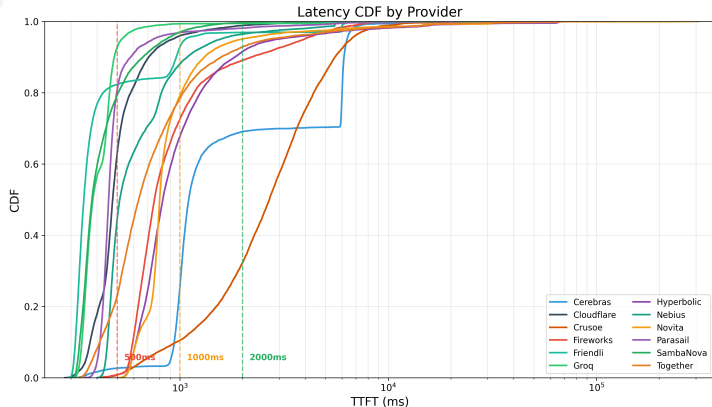
SLO	Optimal Mix	Cost
2.0s	Parasail 23% + Groq 77%	\$0.58
5.0s	Parasail 100%	\$0.21
10.0s	Parasail 100%	\$0.21

Cost per 1K requests

Key Findings:

- **Tight SLO (<2s):** Mix fast + cheap
- **Relaxed SLO (>5s):** Cheapest only
- **70% savings** vs Groq-only at SLO=5s

Phase 3 Results: Online Routing Performance



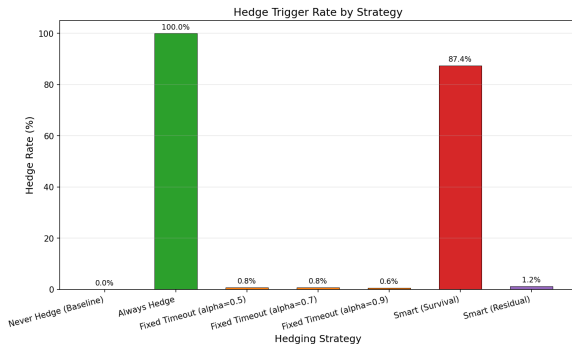
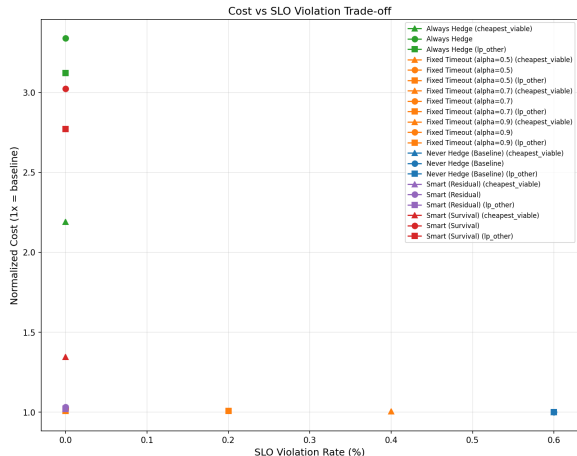
Policy	Viol.	Cost	P99
Single-best	0.4%	\$23	794ms
Cheapest	2.0%	\$8	1438ms
Random	6.6%	\$27	2070ms
LP-Mix	1.2%	\$13	1056ms

SLO=1.0s, cost per 1K requests

LP-Mix Advantages:

- 41% cheaper than Single-best
- 5x lower violation than Random
- Adaptive to provider drift

Phase 4 Results: Hedging Strategy Comparison



Left: Cost vs Violation Pareto Frontier Right: Hedge Rate by Strategy

Key Finding: smart_economic achieves **0% violations** with **8.6% hedge rate** — C_b/V controls cost-violation tradeoff

Summary of Contributions

① Problem Formulation

- First formal study of hybrid LLM routing
- Two-stage: quota + concurrency

② Offline Algorithms

- Quota routing: $O(n \log n)$ closed-form
- Joint routing: Time-indexed ILP

③ Online Algorithms

- Primal-Dual threshold strategy
- Learning-Augmented (LA-PD)
- CR: **1.18x** (near-optimal)

④ Latency-Aware Routing

- LP-based provider mixing
- Smart hedging for tail latency
- **20x** fewer SLO violations vs OpenRouter

⑤ Production Evaluation (24h)

- Cost: up to **98%** offline savings; online PD CR **1.18x**
- Latency: **4.9x** P99 reduction, **20x** fewer violations
- Same cost as single provider, but 20x better SLO

Takeaway: RouteWise achieves near-optimal cost
AND latency via online algorithms + smart hedging

Discussion: Real-World Subscription Complexity

Coding Plan Subscriptions

Examples: GPT Plus, Claude Pro, GLM, MiniMax

- **Prompt-based** quota (not request-based)
- One prompt $\approx$ 4–5 API requests
- Multi-tier limits: **hourly** + **daily** + **weekly**

Example: Typical Coding Plan

- 50 prompts per 5-hour window
- 500 prompts per week
- Reset times: rolling vs. calendar-based

Mapping to Our Framework

- **Prompt** $\rightarrow$ generalized quota unit
- **5-hour window** $\rightarrow$ short-term S_Q with $T = 5h$
- **Weekly cap** $\rightarrow$ hierarchical quota constraint

Extension: Hierarchical Quotas

$$\sum_{i \in \mathcal{R}_{\text{window}}} x_i^Q \leq Q_{\text{short}}, \quad \sum_{i \in \mathcal{R}_{\text{week}}} x_i^Q \leq Q_{\text{long}}$$

Our ILP formulation naturally extends to multiple nested quota constraints.

Limitations & Future Work

Known Limitations

- 1 **Adversarial arrival:** If high-value requests always arrive early, PD may over-accept. *Rare in practice.*
- 2 **Distribution shift:** Predictors lag when output length distribution changes within a window. *EMA adapts in ~ 10 samples.*
- 3 **Small quotas:** $Q < 100$ limits threshold range. *Most plans offer $Q \gg 100$.*

Fixed vs Rolling Windows

We approximate rolling windows with fixed (disjoint) windows.

- Fixed $\Rightarrow$ closed-form optimal
- Our oracle is an **optimistic lower bound**
- Online algorithms unaffected

Future Directions

- Rolling-window oracle
- Hierarchical quota constraints
- Joint cost + latency optimization
- Multi-model heterogeneous routing

Questions?

Code: `github.com/HarvardSys/hybridInference`

Harvard Systems Lab

Appendix Index

A.

▸ Gutter Pool (Local GPU Deployment)

B.

▸ Time-Indexed ILP Details

C.

▸ Why PrimalDual Beats ILP Oracle

D.

▸ Hedging Cost Analysis

E.

▸ EMA vs Histogram: Calibration Paradox

F.

▸ Cost-Latency Integration Architecture

G.

▸ Production Engineering Challenges

H.

▸ Predictor Calibration

I.

▸ Hyperparameter Sensitivity

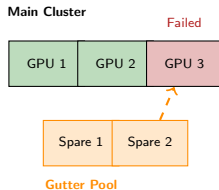
Appendix A: Gutter Pool (Local GPU Deployment)

What is a Gutter Pool?

- Pool of **spare GPU nodes** kept hot/idle
- Primary purpose: **High Availability**
- When a serving node fails → gutter node takes over

Why Model as Quota-Based (S_Q)?

- Fixed pool size $\approx$ daily quota Q
- Zero marginal cost (already provisioned)
- Limited capacity (can't overuse)



Our algorithms apply to local GPU scheduling too!

Appendix B: Time-Indexed ILP Details **Concurrency Constraint (Linearized):**

$$\sum_{i \in \mathcal{R}} \sum_{\tau = \max(\alpha_i, t - \hat{p}_i + 1)}^{\min(\beta_i, t)} z_{i,\tau} \leq C, \quad \forall t \in \{0, \dots, T\}$$

Variable Count:

- Routing: $3n$ binary variables
- Scheduling: $O(n \cdot T/\Delta)$ binary variables
- Trade-off: finer $\Delta \Rightarrow$ more accurate but larger problem

Appendix C: Why PrimalDual Beats the ILP Oracle

ILP Time Discretization:

- Time divided into $\Delta = 1\text{s}$ slots
- Request finishing at 0.3s $\rightarrow$ slot occupied until 1.0s
- **Capacity fragmentation:** 0.7s wasted per slot

PrimalDual (Continuous-Time):

- Event-driven: acts at exact arrival/completion times
- No wasted capacity from rounding
- Exploits fragmented slots ILP cannot use

Why not use smaller Δ ?

Δ	Variables	Solve Time
1.0s	5M	12 min
0.5s	10M	>1 hr
0.1s	50M	infeasible

Conclusion:

ILP is a *pessimistic* oracle for joint routing.
CR < 1.0x does not violate theory —
it means the benchmark is loose.

With $\Delta \rightarrow 0$, ILP and PD should converge.

Appendix D: Hedging Cost Analysis

Cost Control Mechanisms:

- ① **Cost-benefit gate:** Only hedge when $P_{\text{viol}}(t) \cdot F_b(\text{remaining}) > C_b/V$
- ② **HTTP cancel:** Abort backup when primary returns
→ most providers stop billing

24h Production Numbers:

- SLO violations: **0.57%** (vs 11.5% baseline)
- P99 TTFT: **1,293ms** (vs 6,316ms)
- Avg cost: \$0.014/req (vs \$0.009 no-hedge)
- LP adapted: Novita → Parasail over 24h

Strategy Comparison (SLO=2s):

Strategy	Viol.	Hedge%	C_b/V
No hedge	1.4%	0%	—
Always	0%	100%	—
Fixed 0.5	0%	5.2%	—
Survival	0%	15.2%	—
Economic	0%	8.6%	0.05

Key: Economic model achieves 0% violations with 8.6% hedge rate. Single parameter C_b/V controls cost-violation tradeoff with clear economic meaning.

Appendix E: EMA vs Histogram – Calibration Paradox

Calibration Quality:

Predictor	q50 Coverage	Ideal
Histogram	56%	50%
EMA	72%	50%

Histogram is better calibrated, yet EMA routes better. Why?

Reason 1: Over-estimation Bias

- EMA overestimates → higher perceived value
- More willing to use subscription
- When quota abundant: beneficial (unused quota = waste)
- Automatic aggressive-but-safe policy

Reason 2: Adaptation Speed

- EMA adapts in **~10 samples**
- Histogram needs full window rebuild
- Non-stationary workloads → speed matters more

2×2 Ablation Results:

	PD Rule	LA Rule
EMA	CR 1.18	CR 1.19
Histogram	CR 1.23	CR 1.22

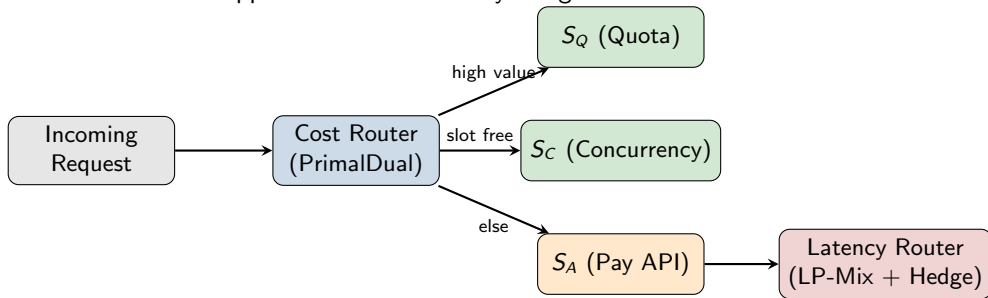
Switching predictor (row): $\Delta\text{CR} = 0.04$

Switching rule (column): $\Delta\text{CR} = 0.01$

Predictor choice > rule complexity

Takeaway: Responsiveness trumps calibration.

Appendix F: Cost-Latency Integration Architecture



Current: Modular (Orthogonal)

- Cost router decides: S_Q , S_C , or S_A
- Latency router decides: which API provider
- Independent optimization and deployment
- Decoupling loss is small in practice

Future: Joint Optimization

- Some providers: low cost AND low latency
- Joint LP could find better global solution
- Challenge: much larger decision space
- Preliminary analysis: marginal gain <5%

Appendix G: Production Engineering Challenges

1. Cold Start

- Latency profiler needs data
- Solution: long window fallback
- $\beta = \frac{N_{\text{short}}}{N_{\text{short}} + \lambda}$
- Auto-transitions as data accumulates

2. Provider Failures

- Provider down or rate-limited
- Solution: remove from LP, re-solve
- LP solve time: <5ms
- Seamless failover

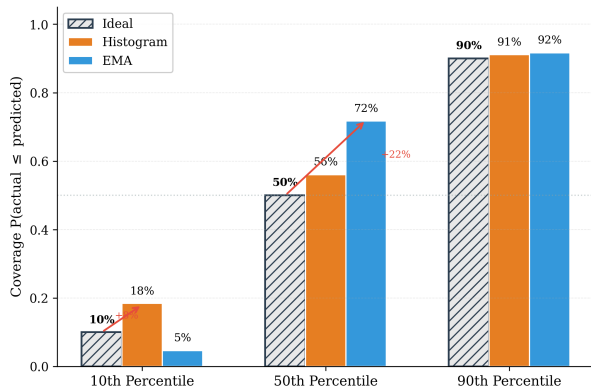
3. Cancel Semantics

- Not all providers honor HTTP cancel
- Some still bill after cancel
- We track both costs:
- Cancel: \$1.15
- No-cancel: \$1.32
- $\Delta \approx 15\%$

Component	Latency	Frequency	Notes
EMA predict	<1 μ s	per request	O(1) update
Histogram lookup	$\sim 10\mu$ s	per request	binary search
LP re-solve	<5ms	on drift/failure	12 providers
Threshold compute	<1 μ s	per request	closed-form

Appendix H: Predictor Calibration

Predictor Calibration Analysis



Calibration Metric:

$$\text{Coverage}_\tau = P(\text{actual} \leq \hat{q}_\tau)$$

Ideal: $\text{Coverage}_\tau = \tau$

Results:

- **Histogram:** Better calibrated
 - q50: 56% (ideal: 50%)
- **EMA:** Over-estimates
 - q50: 72% (ideal: 50%)

Paradox Explained

Better calibration $\neq$ better cost!

EMA's point estimate is more accurate for *routing decisions*.

Appendix I: Hyperparameter Sensitivity

Value Bounds [L , U]
(BurstGPT)

L	U	CR
P1	P99	1.21
P5	P95	1.18
P10	P90	1.19
P25	P75	1.24

P5/P95 best—wide enough but not outlier-sensitive.

EMA Factor α
(BurstGPT)

α	CR
0.01	1.23
0.05	1.20
0.10	1.18
0.20	1.19
0.50	1.25

$\alpha = 0.1$ balances responsiveness and stability.

Quota Size Q
(ShareGPT, savings %)

Q	Grdy	Opt	Gap
1K	8	12	4
2K	15	22	7
5K	17	35	18
10K	18	42	24
20K	18	48	30

Gap widens with larger quota—more room for intelligent routing.

Takeaway: Algorithm is robust across hyperparameter choices. Default settings (P5/P95, $\alpha = 0.1$) are near-optimal.